

Unit - 1

DATE _____
PAGE 1

Question : 1 Explain inheritance & its types with program.

Answer : 1 Java, inheritance is an important pillar of OOP (Object-Oriented Programming). It is the mechanism in Java by which one class is allowed to inherit the features (fields and methods) of another class. Inheritance means creating new classes based on existing ones. A class that inherits from another class can reuse the methods and fields of that class. In addition, you can add new fields and methods to your current class as well.

"Types of inheritance in Java"

1. Single inheritance
2. Multilevel inheritance
3. Hierarchical inheritance
4. Multiple inheritance
5. Hybrid inheritance

1. Single inheritance

In single inheritance, subclasses inherit the features of one superclass. In this inheritance class A, serves as a base

Base
class

A

B

Derived
1

C

Derived
2

D

Derived
3

DATE
PAGE 5

example :

```
class A {
```

```
public void print_A () {
```

```
System.out.println("class A"); }
```

```
class B extends A {
```

```
public void print_B () {
```

```
System.out.println("class B"); }
```

```
class C extends A {
```

```
public void print_C () {
```

```
System.out.println("class C"); }
```

```
class D extends A {
```

```
public void print_D () {
```

```
System.out.println("class D"); }
```

```
public class Test {
```

```
public static void main (String [] args) {
```

```
System.out.println ("for"); }  
class three extends two {  
    public void print_geek () {  
        System.out.println ("geeks"); }  
}
```

```
public class Main {  
    public static void main (String [] args) {  
        three g = new three ();  
        g.print_geek ();  
        g.print_for ();  
        g.print_geek ();  
    }  
}
```

Output :
 for
 geeks
 geeks

3. Hierarchical inheritance : in hierarchical inheritance, one class serves as a superclass (base class) for more than one subclass. A class serves as a base class for the derived classes B, C and D.

output :

Salary : 60000

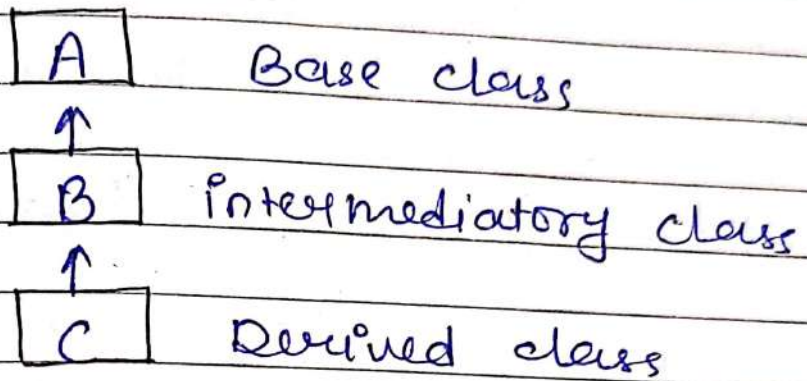
Benefits : 10000

PAGE
PAGE

3

Multilevel inheritance :

2. In multilevel inheritance, a derived class will be inheriting a base class, and as well as the derived class also acts as the base class for another classes.



Example :

```
import java.io.*;  
import java.lang.*;  
import java.util.*;
```

```
class one {
```

```
    public void printGeek () {
```

```
        System.out.println ("Geek");  
    }  
}
```

```
class two extends one {
```

```
    public void printFor () {
```

base class for the derived class B

A



B

Single inheritance

Example : `Import java.io.*;`

```
class Employee {
    int salary = 60000;
}
```

```
class Engineer extends Employee {
    int benefits = 10000;
}
```

```
class Infg {
    public static void main (String args[]) {
```

```
        Engineer E1 = new Engineer();
```

```
        System.out.println ("salary : " + E1.salary
                               + "\n Benefits : " + E1.Benefits);
```

```
    }
```

```
}
```

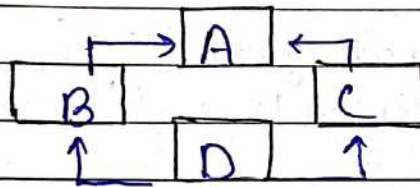
Output :

Creeks

for

Creeks

5. Hybrid inheritance : It is a mix of two or more of the above types of inheritance. Since java doesn't support multiple inheritance with classes, hybrid inheritance involving multiple inheritance is also not possible with classes. In java we can achieve hybrid inheritance only through interfaces. If we want to involve multiple inheritance to implement Hybrid inheritance.



```
public class solarSystem {  
}
```

```
class Earth extends SolarSystem {  
}
```

```
class Mars extends SolarSystem {  
}
```

example 6

```
import java.io.*;
import java.lang.*;
import java.util.*;
```

```
interface one {
    public void print-fox (); }
```

```
interface two {
    public void print-geek (); }
```

```
interface three extends one, two {
    public void print-geek (); }
```

```
class child implements three {
```

```
    public void print-geek () {
        System.out.println ("geeks"); }
```

```
    public void print-fox () {
        System.out.println ("fox"); }
```

```
    public class Main {
        public static void main (String[] args) {
```

```
            child c = new child ();
```

```
            c.print-geek ();
```

```
            c.print-fox ();
```

```
            c.print-geek ();
```

```
        }
    }
```

```
B obj-B = new B ();
obj-B.print-A ();
obj-B.print-B ();
```

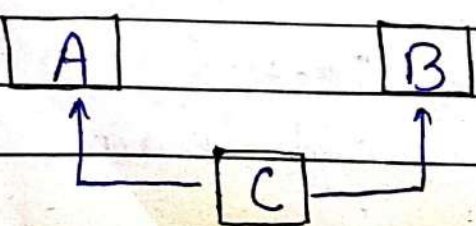
```
C obj-C = new C ();
obj-C.print-A ();
obj-C.print-C ();
```

```
D obj-D = new D ();
obj-D.print-A ();
obj-D.print-D ();
```

Output :

```
class A
Class B
class A
class C
class A
class D
```

4. Multiple inheritance : In this inheritance one class have more than one superclass and inherit features from all parent classes. java does not support multiple inheritance with classes. It can only achieved by interfaces.



Multiple inheritance

6. Use Cases :

Abstract classes are beneficial in scenarios where you have a set of classes sharing common methods or attributes.

7. Example in Java :

```
abstract class Shape {  
    int x, y;  
    abstract void draw();  
    void move() {  
        System.out.println("move shape to x = "  
            + x + " y = " + y);  
    }  
}
```

Conclusion : Abstract classes are a powerful tool for designing class hierarchies by providing a common structure and ensuring that derived classes adhere to a specific set of rules.

The derived classes (subclasses) inherit the properties and behaviours of the abstract class.

3. partial implementation :

Abstract classes can also contain methods with a complete implementation.

This enables the abstract class to provide a default behaviour that subclasses can choose to override.

4. Cannot be instantiated :

You cannot create an instance of an abstract class. It exists solely for other classes to inherit from.

5. Common functionality :

Abstract classes are useful when you want to define a common interface for a group of related classes.

They help in promoting code reusability by providing a common foundation.

Question: 1 Explain use of abstract class in detail?

Answer: 2 An abstract class is object oriented programming serves as a blueprint for other classes. It cannot be instantiated on its own and often contains abstract methods, which are methods without a body. The purpose of an abstract class is to provide a common base for all the classes that inherit from it.

Key points of abstract classes:

1. Abstract Methods:

Abstract classes may have abstract methods, which are declared without providing an implementation.

Subclasses inheriting from an abstract class must implement all its abstract methods.

2. Inheritance:

Abstract classes support inheritance, allowing other classes to derive from them.

```
public class Moon extends Earth {
public static void main (String args[]) {
```

```
SolarSystem s = new SolarSystem ();
Earth e = new Earth ();
Mars m = new Mars ();
```

```
System.out.println (s instead of SolarSystem);
System.out.println (e instead of Earth);
System.out.println (m instead of SolarSystem);
```

Output:

```
true
true
true
```

Question: 4 Difference between class and objects

Answer: 4 There are many difference between object and class.

Class

Objects

- class is used as a template for declaring and creating the objects

An object is an instance of a class

- When a class is created to memory is allocated

objects are allocated memory space whenever they are created

- The class has to be declared first and only once

An object is created many times as per requirement

- A class can not be manipulated as they are not available in the memory

objects can be manipulated

• A class is an

• It is the

• class contains which with

• A class is bind as as

3. The 'finally' block contains code that will be executed regardless of whether an exception occurred or not. It's often used for cleanup activities.

4. If an exception occurs in the 'try' block, the control is transferred to the appropriate 'catch' block. If no exception occurs, the 'catch' block is skipped.

In this example, attempting to divide by zero will trigger an 'ArithmeticException' and the program will gracefully handle this error by printing a specific error message. The 'finally' block ensures that certain code (in this case the print statement) is executed regardless of whether an exception occurred or not.

finally {

System.out.println("This block always
executes"); } }

```
static int divide (int a, int b) {  
    return a/b; } }
```

Explanation:

1. The 'Try' block contains the code that may throw an exception. In this case, the 'divide' method is attempting to perform a division that might result in an 'ArithmeticException' if the divisor ('b') is zero.

2. The 'catch' block specifies how to handle a specific type of exception. It catches an 'ArithmeticException' and prints an error message.

Question : 3 Define Exception handling with suitable example

Answer : 3 Exception handling is a programming concept that deals with the occurrence of runtime errors, allowing the program to gracefully handle unexpected situation it involves catching, handling and possibly recovering from exceptions (error) that may arise during the execution of a program

Example :

```
public class ExampleExceptionHandling {  
    public static void main (String[] args) {  
        try {  
            int result = divide (10, 0);  
            System.out.println ("Result" + result);  
        }  
        catch (ArithmeticException e) {  
            System.out.println ("Error: Division by zero  
is not allowed");  
        }  
    }  
}
```

Information about the HTML document such as character set, viewport settings and the document title

- The `<body>` element contains the content of the HTML document

Text formatting:

- Headings are defined using `<h1>` to `<h6>` tags
- Paragraphs are defined using `<p>` tags
- The `<u>` tag is used for underlined text
- The `<sub>` and `<sup>` tags are used for subscript and superscript
- The `<br>` tag creates a line break
- The `<pre>` tag defines preformatted text, preserving both spaces and line breaks.

<p> This is a paragraph of text </p>
 <p> you can use <u> underline </u> </p>
 <p> This is a _{subscript} and
 this is a ^{superscript} </p>

<p> This is a line break
 It continues on the next line </p>

<p> This is a preformatted text: </p>

```
<pre>
  This is line 1
  This is line 2
  This is line 3
```

</pre>

</body>

</HTML>

Explanation :

- The <!DOCTYPE html> declaration defines the document type and version of HTML.
- The <html> element is the root element of the html document.
- The <Head> elements contains meta-

Unit = 2

DATE 18
PAGE 18

Question : 1 Define HTML structure and demonstrate text formatting

Answer : 1 HTML (Hypertext Markup Language) is the standard markup language for creating web pages. It consists of a series of elements that you use to structure content on the web. HTML elements are represented by tags, and the structure of an HTML document typically includes a head and a body.

HTML structure and its formatting

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<title> Text Formatting Example </title>
```

```
</head>
```

```
<body>
```

```
<h1> This is heading </h1>
```

• A class is a logical entity

• It is declared with the class keyword

• class does not contain any values which can be associated with the field.

• A class is used to bind data as well as methods together as a single unit

An object is a physical entity it is created with a class

It is created with name in C++ and with the 'new' keyword in java

Each object has its own values, which are associated with it

Objects are like a variable of the class

<dt> CSS </dt>
<dd> Cascading style sheets </dd>
</dl>

Output :

- HTML Hyper text mark up language
- CSS Cascading style sheets

These lists can also be nested within each other to create more complex structures. For example you can have an ordered list inside an unordered list.

order, typically with bullet points

- each list item is represented by the `<li>` tag

`<ul>`

`<li> Apple </li>`

`<li> Orange </li>`

`<li> Banana </li>`

`</ul>`

Output :

- Apple
- Orange
- Banana

3. Definition List (`<dl>`):

A definition list is used to represent terms and their corresponding definitions

It consists `<dt>` (definition term) for the term and `<dd>` (definition description) for the definition

`<dl>`

`<dt> HTML </dt>`

`<dd> Hyper text markup language`

Question : 2 How many types of list available in html.
explain with suitable example

Answer : 2 There are three main types of list

1. Ordered List () :
An ordered list is used to represent a list of items in a specific order, typically with numbers.
Each list item is represented by the tag

 First item
 Second item
 Third item

output :

1. first item
2. second item
3. third item

2 Unordered list () :
An unordered list is used to represent a list of items in no particular

Question : 4

HTML table tags

Create a table using its attributes

Answer : 4

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<title> HTML Table Example </title>
```

```
</head>
```

```
<body>
```

```
<h2> Sample Table </h2>
```

```
<table border="1" cellpadding="10" cellspacing="0">
```

```
<thead>
```

```
<tr>
```

```
<th> Name </th>
```

```
<th> Age </th>
```

```
<th> City </th>
```

```
</tr>
```

```
</thead>
```

```
<body>
```

```
<tr>
```

```
<td> John doe </td>
```

cellspacing : cellspacing is another attribute used in the `<table>` tag.

- it specifies the amount of space between cells in a table

(iii) Rowspan and Colspan

- The Rowspan attribute is used to determine the number of cell rows of the table this cell should span. In simple words, it is used to make a cell span more than one row.

- The colspan attribute is used to determine the number of cell columns of the table this cell should span. In simple words, it is used to make a cell span more than one column or used to join columns together.

Question : 9
HTML table
Answer : 9

`<!DOCTYPE`

`<html`

`<head`

`<title`

`</head`

`<body`

`<table`

`tr`

`<tbody`

`<`

- `<u>` Represent underlined text
- `<sub>` Represent subscript text
- `<sup>` Represent superscript text
- `<br>` Represent a line break
- `<pre>` Represent preformatted text

3 List tags:

- `<ul>` Represent an unordered list
- `<ol>` Represent an ordered list
- `<li>` Represent a list item
- `<dl>` Represent a definition list
- `<dt>` Represent a definition item within definition list
- `<dd>` Represent a definition description within a definition list

(i4) Cellpadding & Cellspacing

Cellpadding :

- Cell padding is an attribute used in the `<table>` tag in HTML
- it specifies the amount of space between the content of a cell and its border

Question : 3 Differentiate the following

- i) Tag & attributes
- ii) Callpadding & Callspacing
- iii) Rowspan & Call span

Answer : 3 (i) Tag & attributes

1 Basic structure Tags :

- `<html>` Represents the root element of an html document
- `<head>` contains meta-information about the html document
- `<title>` Sets the title of the HTML document
- `<body>` contains the content of the html document

2 Text Formatting tags :

- `<h1>` to `<h6>` headings from the largest (`h1`) to the smallest (`h6`)
- `<p>` Represents a paragraph
- `<strong>` Represents strong importance (typically bold)

- `<u>` Rep
- `<sub>` R
- `<sup>` Rep
- `<br>` R
- `<pre>` R

3 List

- `<ul>`
- `<ol>`
- `<li>`
- `<dl>`
- `<dd>`

• `<dd>`

(i) K

Cell

used

• it bet

type = "video/mp4" >
your browser does not support
video tag.

</video>

2. Anchor tag :

 Jump to section

<h2 id = "section2" > section 2 </h2>

<p> This is the content of section 2 </p>

3. Frame tag

<!DOCTYPE html>

<html>

<head>

<title> Frameset Example </title>

</head>

<frameset cols = "25% , 75%" >

<frame src = "frame1.html" name = "frame1">

<frame src = "frame2.html" name = "frame2">

Question : 5 Write a short note on following

1. Image tag / video
2. Anchor tag
3. Frame tag
4. form tag

Answer : 5

1. Image tag / video :

- The `<img>` tag is self closing and requires the 'src' attributes to specify the source (URL) of the image

```

```

- Video (`<video>`)

The `<video>` tag is used to embed videos you can specify the source of the video using the `<source>` tag within the `<video>` tag.

```
<video width="640" height="360" controls>
```

```
<source src="path/to/your/video.mp4">
```

```
<td> 25 </td>
<td> New York </td>
</tr>
</tbody>
</table>
</body>
</html>
```

output :

Name	Age	city
John Doe	25	New York
Jane Smith	30	London

Question : 5

1. Image
2. An
3. Fu
4. f

Answer : 5

1. Image

- The requires the so

- Video

The
you
wide
th

<

<

</frameset>
</html>

4. Form tag :

<!DOCTYPE html>

<html lang="en">

<head>

<title> Sample form </title>

</head>

<body>

<h2> Contact us </h2>

<form action="/submit-form" method="post">

<label for="name"> name </label>

<input type="text" id="name" name="name" required>

<label for="message"> message </label>

<textarea id="message" name="message" rows="4" required> </textarea>

<input type="submit" value="Submit">

</form>

</body>

</html>